- **print(object, end='\n', sep=' ')**
- **Escape Charaters:**
  - `\n` : New Line
  - `\t` : Tab
  - `\\` : Backslash
  - `\'` : Single Quote
  - `\"` : Double Quote
- **Indentation**

In [ ]:
```
print(5)
```

```
5
```

In [ ]:
```
print('AMIT Learning')
```

```
AMIT Learning
```

In [ ]:
```
print('Ahmed',  'Ali', sep='||')
```

```
Ahmed||Ali
```

In [ ]:
```
print('Hello', end='----')



print('AI')
print('DS')
```

```
Hello----AI
DS
```

In [ ]:
```
print('AHmed', 22)
```

In [ ]:
```
fname = 'ahmed'
age = 22
```

In [ ]:
```
print(fname, age)
```

```
ahmed 22
```

In [ ]:
```
x2 = 10
```

In [ ]:
```
x& = 22
```

```
  Cell In[16], line 1

    x& = 22

       ^

SyntaxError: invalid syntax
```

In [ ]:
```
x_ = 25
```

In [ ]:
```
X = 5
x = 4
```

In [ ]:
```
print(X)
print(x)
```

```
5
4
```

In [ ]:
```

```

Variables

In [ ]:
```

```

Datatypes

- **Text Type:** `str`
- **Numeric Types:** `int`, `float`
- **Boolean Type:** `bool`
- **Sequence Types:** `list`, `tuple`, `set`
- **Mapping Types:** `dict`

`type()`

In [ ]:
```
name1 = 'ahmed'
name2 = 'omar'
name3 = 'ali'
```

In [ ]:
```
names = ['ahmed', 'ali', 'omar', 'amr']
```

In [ ]:
```
name = 'ali'
print(type(name))
```

```
<class 'str'>
```

In [ ]:
```
age = 25
print(type(age))
```

```
<class 'int'>
```

In [ ]:
```
h = 1.77
print(type(h))
```

```
<class 'float'>
```

Strings

- **List Functions & Methods**
  - `len()` : Returns the length of a string.
  - `str()` : Convert an object to a string representation.
  - `sorted()` : Sorts characters in a string and returns it as a new string.
  - `max()` , `min()` : Return the maximum or minimum value from a string.
  - `.lower(), .upper()` : Convert all characters in a string to lowercase or uppercase, respectively.
  - `.capitalize()` : Converts the first character in a string to capital letters.
  - `.title()` : Convert the first character in each word in astring to capital letters.
  - `.find(substring)` : Returns the lowest index in the string where substring is found. If not found, returns -1.
  - `.count()` : Count how many times a substring appears in a string.
  - `.replace('substring', 'substring')` : Replace occurrences of one substring with another.
  - `.format()` , `f"{}"` : Format a string using placeholders and values provided by curly braces.

In [ ]:
```
# casting
x = 25
y = str(x)
print(type(x), type(y))
```

```
<class 'int'> <class 'str'>
```

In [ ]:
```
txt = 'Hello in AI and Data Science Diploma at AMIT'
txt = "Hello in AI and Data Science Diploma at AMIT"
txt = """Hello in AI and Data Science Diploma at AMIT"""
txt = '''Hello in AI and Data Science Diploma at AMIT'''
```

In [ ]:
```
print("Ahmed's car")
```

```
Ahmed's car
```

In [ ]:
```
print('Ahmed said "Hello AI"')
```

```
Ahmed said "Hello AI"
```

In [ ]:
```
par = """Hello in AI and Data Science
Diploma at
AMIT"""
```

In [ ]:
```
txt
```

```
'Hello in AI and Data Science Diploma at AMIT'
```

In [ ]:
```
txt = 'Hello in AI and {} Diploma at AMIT'
round = 25
print(txt.format(round))
```

```
Hello in AI and 25 Diploma at AMIT
```

In [ ]:
```
txt = f'Hello-in-AI-and-{round}-Diploma-at-AMIT'
print(txt)
```

```
Hello-in-AI-and-25-Diploma-at-AMIT
```

In [ ]:
```
print(txt.split('-'))
```

```
['Hello', 'in', 'AI', 'and', '25', 'Diploma', 'at', 'AMIT']
```

In [ ]:
```
s = ['Hello', 'in', 'AI', 'and', '25', 'Diploma', 'at', 'AMIT']
t = '||'.join(s)
print(t)  # Output: Hello in AI and 25 Diploma at AMIT
```

```
Hello||in||AI||and||25||Diploma||at||AMIT
```

In [ ]:
```
print(txt.replace('at', 'in', 2))
```

```
Hello in in and at Diploma at AMIT
```

In [ ]:
```
print(txt)
print(txt.upper())
print(txt.lower())
print(txt.capitalize())
print(txt.title())
print(len(txt))
print(txt.find('a'))
print(txt.count('AI'))
print(txt.replace('in', '25'))
```

```
Hello in AI and AI Diploma at AMIT
HELLO IN AI AND AI DIPLOMA AT AMIT
hello in ai and ai diploma at amit
Hello in ai and ai diploma at amit
Hello In Ai And Ai Diploma At Amit
34
12
2
Hello 25 AI and AI Diploma at AMIT
```

Strings

- **String Functions & Methods**
  - `len()` : Returns the length of a string.
  - `str()` : Convert an object to a string representation.
  - `sorted()` : Sorts characters in a string and returns it as a new string.
  - `max()` , `min()` : Return the maximum or minimum value from a string.

- `.lower()`, `.upper()` : Convert all characters in a string to lowercase or uppercase, respectively.
- `.capitalize()` : Converts the first character in a string to capital letters.
- `.title()` : Convert the first character in each word in astring to capital letters.
- `.find(substring)` : Returns the lowest index in the string where substring is found. If not found, returns -1.
- `.count()` : Count how many times a substring appears in a string.
- `.replace('substring', 'substring')` : Replace occurrences of one substring with another.
- `.format()` , `f"{}"` : Format a string using placeholders and values provided by curly braces.

In [ ]:
```
text = "Hello, World!"
print(text.capitalize())
```

```
Hello, world!
```

In [ ]:
```
s = "  Hello AMIT  "
print(s.lstrip())
```

```
Hello AMIT
```

In [ ]:
```
s = 'I-love-Machine-Learning,-and-I-want-to-be-a-Machine-Learning-Engineer'
words = s.split('-')
print(words)
```

```
['I', 'love', 'Machine', 'Learning,', 'and', 'I', 'want', 'to', 'be', 'a', 'Machin
e', 'Learning', 'Engineer']
```

In [ ]:
```
words
```

```
['I',
 'love',
 'Machine',
 'Learning,',
 'and',
 'I',
 'want',
 'to',
 'be',
 'a',
 'Machine',
 'Learning',
 'Engineer']
```

In [ ]:
```
" ".join(words)
```

```
'I love Machine Learning, and I want to be a Machine Learning Engineer'
```

In [ ]:
```
print(text.startswith('Hello'))
```

```
True
```

In [ ]:
```
name = 'Ali'
msg = f"Hello {name}, how are you?"
```

```
print(msg)
```

```
Hello Ali, how are you?
```

In [ ]:
```
print(text.find('W'))
```

```
7
```

In [ ]:
```
text = "Hello, World!"
print(len(text))
```

In [ ]:
```
num = 123
string_num = str(num)
print(string_num)
print(type(string_num))
```

In [ ]:
```
text = "python"
sorted_text = sorted(text)
print(sorted_text)
```

In [ ]:
```
text = "abcdef"
print(max(text))
print(min(text))
```

In [ ]:
```
text = "Hello, World!"
print(text.lower())
print(text.upper())
```

In [ ]:
```
text = "hello, world!"
print(text.capitalize())
```

In [ ]:
```
text = "hello world, welcome to python!"
print(text.title())
```

In [ ]:
```
text = "hello world"
index = text.find("world")
print(index)

not_found = text.find("python")
print(not_found)
```

In [ ]:
```
text = "banana"
count = text.count("a")
print(count)
```

In [ ]:
```
text = "I love apples"
new_text = text.replace("apples", "bananas")
print(new_text)
```

In [ ]:
```
name = "Omar"
age = 25
formatted_text = "My name is {} and I am {} years old.".format(name, age)
print(formatted_text)
```

In [ ]:
```
name = "Ali"
age = 30
formatted_text = f"My name is {name} and I am {age} years old."
print(formatted_text)
```

Numeric Types

- `int` : integer, positive or negative with unlimited length.
- `float` : floating point number, can represent fractional numbers.

`casting functions`

In [ ]:
```
x = 2.616548949
print(round(x, 1))
```

```
2.6
```

In [ ]:
```
x = 5
y = -6
z = 3.14
a = -122.53
```

In [ ]:
```
print(type(x))
print(type(y))
print(type(z))
print(type(a))
```

Operators and Expressions

- Assignment operators: `=` , `+=` , `-=` etc.
- Arithmetic operators (`+` , `-` , `*` , `/` , `**` , `//` , `%`) work with numeric types, except `+` it can work with strings as well.
- Comparison operators (`==` , `<` , `<=` , `>` , `>=`) work with any data type that supports them (e.g., text vs numbers).
- Logical operators: `&&` , `||` (logical AND, logical OR).

In [ ]:
```
a = 10
a += 5  # Now a is 15
a *= 2  # Now a is 30
print(a)  # Output: 30
```

In [ ]:
```
x = 5
y = 7
x
y
```

```
7
```

In [ ]:
```
x = 15
y = 4

print(x + y)
print(x - y)
print(x * y)
print(x ** y)
print(x / y)
print(x // y)
print(x % y)
```

In [ ]:
```
a = 10
b = 5

print(a == b)
print(a != b)
print(a > b)
print(a <= b)
```

In [ ]:
```
x = 10
y = 20

print(x > 5 and y < 30)
print(x < 5 or y > 15)
print(not (x == y))
```

In [ ]:
```
x = 5
print(x ** 2)
print(pow(x, 2))
```

```
25
25
```

Boolean types: `True`, `False`

In [ ]:
```

```

Sequence Types

`List []`

- A collection of **mutable** items in a particular order.
- Lists are **dynamic** and can grow or shrink as needed.
- Supports efficient insertion/deletion at arbitrary locations.
- Can contain elements of any data type, including custom objects.

- They have **zero-based** index.
- `list()` function is used to create a new list object.

List Functions & Methods

- `len()` : Returns the number of elements in the list.
- `max()` / `min()` : Return the maximum / minimum value from the list.
- `sum()` : Calculate the sum of all values in the list.
- `del()` : Deletes the item at the given index.
- `.append(item)` : Add an element to the end of the list.
- `.extend([items])` : Add multiple items to the end of the list.
- `.insert(pos, item)` : Insert an element at a specific position.
- `.remove(item)` : Remove the first occurrence of a specified value. Raises an error if not found.
- `.pop()` : Remove and return an item at the given position. If no index provided, removes and returns the last item.
- `.clear()` : Removes all items from the list.
- `.copy()` : Returns a copy of the original list.
- `.count` : Count how many times an element appears in the list.
- `.index()` : Find the first occurrence of a value in the list. If not found, raises an error.

In [ ]:
```
data = ['ali', 22, 'cairo', ['AI', 'DS', 'DA']]
data
```

```
['ali', 22, 'cairo', ['AI', 'DS', 'DA']]
```

In [ ]:
```
max(data)
```

```
---------------------------------------------------------------------------

TypeError                                 Traceback (most recent call last)

Cell In[38], line 1

----> 1 max(data)



TypeError: '>' not supported between instances of 'int' and 'str'
```

In [ ]:
```
marks = [90, 85, 70, 91, 91]
print(max(marks))
print(min(marks))
print(sum(marks))
```

```
91
70
427
```

In [ ]:
```
sum(max(marks))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[36], line 1
----> 1 sum(max(marks))


TypeError: 'int' object is not iterable
```

In [ ]:
```
names
```

```
['Ali', 'omar', 'amr', 'ahmed']
```

In [ ]:
```
sum(names)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[32], line 1
----> 1 sum(names)


TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

In [ ]:
```
print('ali' + 'omar')
```

```
aliomar
```

In [ ]:
```
names = ['Ali', 'omar', 'amr', 'ahmed']
print(names)
```

```
['Ali', 'omar', 'amr', 'ahmed']
```

In [ ]:
```
names2 = names.copy()
names2
```

```
['Ali', 'omar', 'amr', 'ahmed']
```

In [ ]:
```
names.append('Aya')
print(names)
```

```
['Ali', 'omar', 'amr', 'ahmed', 'Aya']
```

In [ ]:
```
names.append(['ola', 'khaled', 'abdo'])
print(names)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[9], line 1
----> 1 names.append('ola', 'khaled', 'abdo')
      2 print(names)

TypeError: list.append() takes exactly one argument (3 given)
```

In [ ]:
```
names.extend(['abdullah', 'esraa', 'ziad'])
print(names)
```

```
['Ali', 'omar', 'amr', 'ahmed', 'Aya', ['ola', 'khaled', 'abdo'], 'abdullah', 'esra
a', 'ziad']
```

In [ ]:
```
names.insert(3, 'Huda')
print(names)
```

```
['Ali', 'omar', 'amr', 'Huda', 'ahmed', 'Aya', ['ola', 'khaled', 'abdo'], 'abdulla
h', 'esraa', 'ziad']
```

In [ ]:
```
names.remove('Aya')
print(names)
```

```
['Ali', 'omar', 'amr', 'Huda', 'ahmed', ['ola', 'khaled', 'abdo'], 'abdullah', 'esra
a', 'ziad']
```

In [ ]:
```
names.pop(5)
print(names)
```

```
['Ali', 'omar', 'amr', 'Huda', 'ahmed', 'abdullah', 'esraa', 'ziad']
```

In [ ]:
```
names.clear()
print(names)
```

```
[]
```

In [ ]:
```
names = ['Ali', 'omar', 'amr', 'Huda', 'ahmed', 'abdullah', 'esraa', 'ziad']
del(names)
print(names)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[16], line 3
      1 names = ['Ali', 'omar', 'amr', 'Huda', 'ahmed', 'abdullah', 'esraa', 'ziad']
      2 del(names)
----> 3 print(names)

NameError: name 'names' is not defined
```

In [ ]:
```
names = ['Ali', 'omar', 'amr', 'omar', 'ahmed', 'abdullah', 'esraa', 'ziad']
names.index('ahmed')
```

```
4
```

In [ ]:
```
print(names.count('omar'))
```

```
2
```

In [ ]:
```
print(len(names))
```

```
4
```

In [ ]:
```
print(sorted(names, reverse=True))
```

```
['omar', 'amr', 'ahmed', 'Ali']
```

In [ ]:
```
s = "AMIT"
print(sorted(s, reverse=True))
```

```
['T', 'M', 'I', 'A']
```

In [ ]:
```
names
```

```
['Ali', 'omar', 'amr', 'ahmed']
```

In [ ]:
```
names[1]
```

```
'omar'
```

`Tuple ()`

- Immutable, ordered collection of items.
- Cannot be modified after creation.
- Useful for fixed-size collections where the order of elements matter and you don't want them changed accidentally.
- Tuples can be used to group related data together. For example, **coordinates** (x, y) or **RGB** color values (red, green, blue).
- Can contain elements of any data type, including custom objects.

- They have zero-based.
- `tuple()` function is used to create a new tuble object.

List Functions & Methods

- `len()` : Returns the number of elements in the tuple.
- `max()` / `min()` : Return the maximum / minimum value from the tuple.
- `sum()` : Calculate the sum of all values in the tuple.
- `.count()` : Count how many times an element appears in the tuple.
- `.index(value)` : Find the index position of a specific item within the tuple.

In [ ]:
```
t = (25, 6, 44)
print(type(t))
```

```
<class 'tuple'>
```

In [ ]:
```
t.append(8)
```

```
---------------------------------------------------------------------------

AttributeError                            Traceback (most recent call last)

Cell In[40], line 1

----> 1 t.append(8)



AttributeError: 'tuple' object has no attribute 'append'
```

In [ ]:
```
t[0] = 60
```

```
---------------------------------------------------------------------------

TypeError                                 Traceback (most recent call last)

Cell In[46], line 1

----> 1 t[0] = 60



TypeError: 'tuple' object does not support item assignment
```

In [ ]:
```
names[0] = 'abdullah'
names
```

```
['abdullah', 'omar', 'amr', 'ahmed']
```

In [ ]:
```
c = (220, 100, 60)
```

In [ ]:
```
l = list(c)
l
```

```
[220, 100, 60]
```

In [ ]:
```
l.pop(0)
```

```
220
```

In [ ]:
```
t = tuple(l)
t
```

```
(100, 60)
```

In [ ]:
```
p = (25, 6)
```

`Set {}`

- A collection of elements.
- The elements in a set are not stored in any particular order.
- Sets cannot contain duplicate elements.
- Can contain elements of any data type, including custom objects.
- They are not subscriptable, meaning you can't access individual elements using square brackets (`[]`).
- `set()` function is used to create a new set object.

List Functions & Methods

- `len()` : Returns the number of elements in the set.
- `max()` / `min()` : Return the maximum / minimum value from the set.
- `sum()` : Calculate the sum of all values in the set.
- `del()` : Deletes the set.
- `.add()` :Add an item to the set if not already present.
- `.update()` : Add multiple items to the set.
- `.remove(item)` : Remove a single specified item from the set.
- `.discard()` : Remove a single specified item from the set if it exists.
- `.pop()` : Remove and return an arbitrary item from the set.
- `.clear()` : Removes all items from the set.
- `.copy()` : Create a copy of the original set.
- `issubset()` , `issuperset()` : Check if one set is a subset or superset of another.
- `union()` , `intersection()` , `difference()` : Perform operations on sets.

In [ ]:
```
s1 = {12, 6}
s2 = {12, 44, 6}

s2.issuperset(s1)
```

```
True
```

In [ ]:
```
s1.intersection(s2)
```

```
{12}
```

In [ ]:
```
s2.difference(s1)
```

```
{6, 44}
```

In [ ]:
```
d = {25, 30, 6, 25}
d
```

```
{6, 25, 30}
```

In [ ]:
```
sorted(d, reverse=True)
```

```
[30, 25, 6]
```

In [ ]:
```
d.add(17)
d
```

```
{6, 17, 25, 30}
```

In [ ]:
```
d.clear()
d
```

```
set()
```

In [ ]:
```
d.add(17)
d
```

```
{6, 17, 25, 30}
```

In [ ]:
```
d.add(30)
d
```

```
{6, 17, 25, 30}
```

In [ ]:
```
d.update([25, 4, 9, 27])
d
```

```
{4, 6, 9, 17, 25, 27, 30}
```

In [ ]:
```
names
```

```
['abdullah', 'omar', 'amr', 'ahmed']
```

In [ ]:
```
names.pop()
names
```

```
['abdullah', 'omar', 'amr']
```

In [ ]:
```
d.remove(17)
d
```

```
{4, 6, 9, 25, 27, 30}
```

In [ ]:
```
d.remove(31)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[61], line 1
----> 1 d.remove(31)


KeyError: 31
```

In [ ]:
```
d.discard(31)
```

In [ ]:
```
d.pop()
d
```

```
{6, 9, 25, 27, 30}
```

In [ ]:
```
len(d)
```

```
3
```

`Dictionary {k:v}`

- A collection of **key-value** pairs.
- The key-value pairs in a dictionary are not stored in any particular order.
- Keys in a dictionary must be **unique**, but values can be duplicated.
- Keys can be of any **immutable** data type, such as integers, strings, or tuples. Values can be of any data type, including custom objects.
- Dictionaries are subscriptable using keys ([]) to access individual values.
- `dict()` function is used to create a new dictionary object.

Dictionary Functions & Methods

- `len()` : Returns the number of key-value pairs in the dictionary.
- `del()` : Deletes the item with the specified key.
- `.keys()` : Returns a view object of all keys in the dictionary.
- `.values()` : Returns a view object of all values in the dictionary.
- `.items()` : Returns a view object of all key-value pairs (tuples) in the dictionary.
- `.get()` : Returns the value of the specified key. If the key does not exist, it returns None (or a default value if provided).
- `.pop()` : Removes and returns the value of the specified key. If the key does not exist, it raises a KeyError (or returns a default value if provided).
- `.popitem()` : Removes and returns an arbitrary key-value pair from the dictionary as a tuple.
- `.clear()` : Removes all key-value pairs from the dictionary.
- `.copy()` : Creates a shallow copy of the dictionary.
- `.update()` : Updates the dictionary with the specified key-value pairs from another dictionary or iterable of key-value pairs.

In [ ]:
```
data = ['ahmed', 25, 'cairo', ['AI', 'ML', 'DS']]
data
```

```
['ahmed', 25, 'cairo', ['AI', 'ML', 'DS']]
```

In [ ]:
```
data[0]
```

```
'ahmed'
```

In [ ]:
```
data = {'fname':'ahmed', 'age':25, 'city':'cairo', 'skills':['AI', 'DS', 'ML']}
data
```

```
{'fname': 'ahmed', 'age': 25, 'city': 'cairo', 'skills': ['AI', 'DS', 'ML']}
```

In [ ]:
```
data.keys()
```

```
dict_keys(['fname', 'age', 'city', 'skills'])
```

In [ ]:
```
data.values()
```

```
dict_values(['ahmed', 25, 'cairo', ['AI', 'DS', 'ML']])
```

In [ ]:
```
data.items()
```

```
dict_items([('fname', 'ahmed'), ('age', 25), ('city', 'cairo'), ('skills', ['AI', 'D
S', 'ML'])])
```

In [ ]:
```
data.get('age')
```

```
25
```

In [ ]:
```
data
```

```
{'fname': 'ahmed', 'age': 25, 'city': 'cairo'}
```

In [ ]:
```
data.popitem()
data
```

```
{'fname': 'ahmed', 'age': 25}
```

In [ ]:
```
data.pop('skills')
```

```
['AI', 'DS', 'ML']
```

In [ ]:
```
data
```

```
{'fname': 'ahmed', 'age': 25, 'city': 'cairo'}
```

In [ ]:
```
data['fname']
```

```
'ahmed'
```

In [ ]:
```
print(data)
```

```
{'fname': 'ahmed', 'age': 25, 'city': 'cairo', 'skills': ['AI', 'DS', 'ML']}
```

In [ ]:
```
len(data)
```

```
4
```

In [ ]:
```
data
```

```
{'fname': 'ahmed', 'age': 25}
```

In [ ]:
```
data['city'] = 'cairo'
data
```

```
{'fname': 'ahmed', 'age': 25, 'city': 'cairo'}
```

In [ ]:
```
data.update({'skills':['AI', 'DS', 'ML']})
data
```

```
{'fname': 'ahmed', 'age': 25, 'city': 'cairo', 'skills': ['AI', 'DS', 'ML']}
```

Control Statements

`if..else`

In [ ]:
```
age = 20

if age > 18:
    print('Accept')
else:
    print('Reject')
```

```
Accept
```

In [ ]:
```
l = ["123456", "63589745", "amit", "machine learning"]
```

In [ ]:
```
password = l[-1]
if len(password) >= 8:
    print("Password is strong")

else:
    print("Password is weak")
```

```
Password is strong
```

Loops

- Loops allow you to execute a block of code repeatedly.
- There are two types of loops that we can use inPython:
  - The `for` loop
  - The `while` loop

`For`

A for loop is used when you know how many times you want to execute a block of code.

In [ ]:
```
print('Hello Ahmed')
print("Hello Omar")
print("Hello Ali")
```

In [ ]:
```
names = ["Ali", 'Amr', 'Omar', 'Ola', 'Ahmed']
names[2]
```

```
'Omar'
```

In [ ]:
```
names = ["Ali", 'Amr', 'Omar', 'Ola', 'Ahmed']
print(f"Hello {names[0]}")
print(f"Hello {names[1]}")
print(f"Hello {names[2]}")
print(f"Hello {names[3]}")
print(f"Hello {names[4]}")
```

```
Hello Ali
Hello Amr
Hello Omar
Hello Ola
Hello Ahmed
```

In [ ]:
```
for i in range(1, 5):
    print(i)
```

```
1
2
3
4
```

In [ ]:

In [ ]:
```
for i in "hello":
    print(i)
```

```
h
e
l
l
o
```

In [ ]:
```
for i in range(0, 101, 5):
    print(i)
```

```
0
5
10
15
20
25
30
35
40
45
50
55
60
65
70
75
80
85
90
95
100
```

In [ ]:

```

```

In [ ]:

```
s = "Hello"
for i in range(len(s)):
```

In [ ]:

```
i = 'ali'
i = 'ahmed'
i = 'amr'
```

In [ ]:

```
students_names = ["aLi", 'Ahmed', 'Amr']
for i in students_names:
    print(f"Hello {i}")
```

```
Hello aLi
Hello Ahmed
Hello Amr
```

In [ ]:

```
for name in names:
    print(f"Hello {name}")
```

```
Hello Ali
Hello Amr
Hello Omar
Hello Ola
Hello Ahmed
```

In [ ]:

```
range(5)
```

```
range(0, 5)
```

In [ ]:

```
for i in [0, 1, 2, 3, 4]:
    print(i)
```

```
0
1
2
3
4
```

In [ ]:

In [ ]:

In [ ]:

- **Iterating over a `range` of numbers:**

In [ ]:
```
l = ['ali', 'amr', 'omar']
for i in range(len(l)):
    print(l[i])
```

```
ali
amr
omar
```

In [ ]:
```
for i in l:
    print(i)
```

```
ali
amr
omar
```

In [ ]:
```
for i in range(1, 20, 4):
    print(i)
```

```
1
5
9
13
17
```

In [ ]:
```
l = ['ali', 'amr', 'omar']
for i in range(len(l)):
    print(l[i])
```

```
ali
amr
omar
```

- **Iterating over characters in a `string`:**

In [ ]:
```
s = "Hello"
for i in s:
    print(i)
```

```
H
e
l
l
o
```

- **Iterating over elements in a `list`:**

In [ ]:
```
len(l)
```

```
6
```

In [ ]:
```
l[2]
```

```
89
```

In [ ]:
```
l = [78, 99, 89, 77, 96, 88]
for i in range(len(l)):
    print(l[i])
```

```
78
99
89
77
96
88
```

- **Iterating over `tuples`:**

In [ ]:
```
input()
```

In [ ]:
```
t = ('ahmed', 'ali', 'omar')

for i in t:
    print(i)
```

```
ahmed
ali
omar
```

In [ ]:
```
t = ('ahmed', 'ali', 'omar')
for i in range(len(t)):
    print(f"{i+1} {t[i]}")
```

```
1 ahmed
2 ali
3 omar
```

- **Iterating over key-value pairs in a `dictionary`:**

In [ ]:
```
d = {"Ids":[101, 102, 103], 'Names':['Ali', 'Ahmed', "Amr"], 'Age': [20, 21, 22]}
d
```

```
{'Ids': [101, 102, 103], 'Names': ['Ali', 'Ahmed', 'Amr'], 'Age': [20, 21, 22]}
```

In [ ]:
```
d.items()
```

```
dict_items([('Ids', [101, 102, 103]), ('Names', ['Ali', 'Ahmed', 'Amr']), ('Age', [2
0, 21, 22])])
```

In [ ]:
```
t = ('Ids', [101, 102, 103])
t
```

```
('Ids', [101, 102, 103])
```

In [ ]:
```
k, v = ('Ids', [101, 102, 103])
```

In [ ]:
```
k
```

```
'Ids'
```

In [ ]:
```
v
```

```
[101, 102, 103]
```

In [ ]:
```
for k, v in d.items():
    print(f"{k} ==> {v}")
```

```
Ids ==> [101, 102, 103]
Names ==> ['Ali', 'Ahmed', 'Amr']
Age ==> [20, 21, 22]
```

In [ ]:
```
for i in range(20):
    if i == 12:
        break
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
10
11
```

In [ ]:
```
for i in range(20):
    if i == 12:
        continue
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
10
11
13
14
15
16
17
18
19
```

In [ ]:
```
if age > 20:
    pass
```

In [ ]:
```
if age > 20:
    pass
```

In [ ]:

In [ ]:

In [ ]:
```
for i in range(20):
    if i == 12:
        pass
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
```

In [ ]:

In [ ]:
```python
def test():
    pass
```

In [ ]:

- input
- output (print)
- Variables
- Data types
  - str
  - int
  - float
  - bool
  - list
  - tuple
  - set
  - dictionary
- if..elif..else
- for..loop
- while..loop
- break, continue - pass
- Try..Except

In [ ]:
```python
try:
    num1 = int(input())
    num2 = int(input())

    if num1 % num2 == 0:
        print('Yes')

    else:
        print('No')
except ValueError:
    print('Please enter a valid number')

except ZeroDivisionError:
    print("You can't divide by zero!")
```

```
You can't divide by zero!
```